```
"""
Stock tracker: technical signals, risk-based position sizing, and a backtest.
Not financial advice. Past performance does not predict future results.

Setup:  pip install yfinance pandas numpy
Run:    python stock_tracker.py
        python stock_tracker.py --tickers AAPL NVDA SPY --account 25000 --risk 0.
"""
import argparse

import numpy as np
import pandas as pd
import yfinance as yf

DEFAULT_WATCHLIST = ["AAPL", "MSFT", "NVDA", "AMZN", "GOOGL", "TSLA", "SPY"]


# ---------- indicators ----------
def rsi(close, n=14):
    d = close.diff()
    up = d.clip(lower=0).ewm(alpha=1 / n, adjust=False).mean()
    dn = (-d.clip(upper=0)).ewm(alpha=1 / n, adjust=False).mean()
    return 100 - 100 / (1 + up / dn)


def macd(close):
    line = close.ewm(span=12, adjust=False).mean() - close.ewm(span=26, adjust=Fa
    return line, line.ewm(span=9, adjust=False).mean()


def atr(df, n=14):
    pc = df["Close"].shift()
    tr = pd.concat(
        [df["High"] - df["Low"], (df["High"] - pc).abs(), (df["Low"] - pc).abs()]
    ).max(axis=1)
    return tr.ewm(alpha=1 / n, adjust=False).mean()


def add_indicators(df):
    df = df.copy()
    df["SMA50"] = df["Close"].rolling(50).mean()
    df["SMA200"] = df["Close"].rolling(200).mean()
    df["RSI"] = rsi(df["Close"])
```

```
    df["MACD"], df["MACD_SIG"] = macd(df["Close"])
    df["ATR"] = atr(df)
    df["VOL_RATIO"] = df["Volume"] / df["Volume"].rolling(20).mean()
    df["RET"] = df["Close"].pct_change()
    return df


# ---------- scoring ----------
def score(df):
    """Score each day from -2 to +4 using trend, momentum, and volume."""
    s = pd.Series(0, index=df.index, dtype=float)
    s += ((df["Close"] > df["SMA50"]) & (df["SMA50"] > df["SMA200"])).astype(int)
    s += (df["MACD"] > df["MACD_SIG"]).astype(int)
    s += df["RSI"].between(50, 70).astype(int)
    s += ((df["VOL_RATIO"] > 1.5) & (df["RET"] > 0)).astype(int)
    s -= (df["RSI"] > 75).astype(int)
    s -= (df["Close"] < df["SMA200"]).astype(int)
    return s


def label(sc):
    return "BUY" if sc >= 3 else "WATCH" if sc == 2 else "AVOID"


# ---------- risk ----------
def position_size(price, atr_val, account, risk_pct, atr_mult=2.0):
    """Risk a fixed % of the account on each trade; stop = price - atr_mult * ATR
    stop_dist = atr_mult * atr_val
    shares = int((account * risk_pct) // stop_dist) if stop_dist > 0 else 0
    shares = min(shares, int(account // price))  # no leverage
    return shares, price - stop_dist


# ---------- backtest ----------
def backtest(df, entry_score=3, cost=0.001):
    """Hold when yesterday's score >= entry_score. Compare against buy-and-hold."
    d = df.dropna().copy()
    d["SCORE"] = score(d)
    pos = (d["SCORE"] >= entry_score).astype(int).shift(1).fillna(0)  # trade nex
    trades = pos.diff().abs().fillna(0)
    strat = pos * d["RET"] - trades * cost
    equity = (1 + strat).cumprod()
    bh = (1 + d["RET"]).cumprod()
    dd = (equity / equity.cummax() - 1).min()
    sharpe = strat.mean() / strat.std() * np.sqrt(252) if strat.std() > 0 else 0
    return {
```

```
        "strategy_return": equity.iloc[-1] - 1,
        "buy_hold_return": bh.iloc[-1] - 1,
        "max_drawdown": dd,
        "sharpe": sharpe,
        "time_in_market": pos.mean(),
    }


# ---------- main ----------
def analyze(ticker, account, risk_pct):
    df = yf.Ticker(ticker).history(period="5y", auto_adjust=True)
    if df.empty or len(df) < 250:
        return None
    df = add_indicators(df)
    last = df.iloc[-1]
    sc = int(score(df).iloc[-1])
    shares, stop = position_size(last["Close"], last["ATR"], account, risk_pct)
    bt = backtest(df)
    return {
        "Ticker": ticker,
        "Price": round(last["Close"], 2),
        "RSI": round(last["RSI"], 1),
        "Score": sc,
        "Signal": label(sc),
        "Shares@Risk": shares,
        "Stop": round(stop, 2),
        "BT Return": f"{bt['strategy_return']:.0%}",
        "Buy&Hold": f"{bt['buy_hold_return']:.0%}",
        "MaxDD": f"{bt['max_drawdown']:.0%}",
        "Sharpe": round(bt["sharpe"], 2),
    }


def main():
    p = argparse.ArgumentParser()
    p.add_argument("--tickers", nargs="+", default=DEFAULT_WATCHLIST)
    p.add_argument("--account", type=float, default=10000, help="account size in
    p.add_argument("--risk", type=float, default=0.01, help="fraction of account
    a = p.parse_args()

    rows = []
    for t in a.tickers:
        try:
            r = analyze(t.upper(), a.account, a.risk)
            if r:
                rows.append(r)
```

```
            except Exception as e:
                print(f"{t}: failed ({e})")

    if not rows:
        print("No data returned.")
        return
    out = pd.DataFrame(rows).sort_values("Score", ascending=False)
    pd.set_option("display.width", 200)
    print(out.to_string(index=False))
    print("\nBacktest covers ~5 years, includes 0.1% trading cost, and ignores ta
    print("If 'BT Return' doesn't beat 'Buy&Hold' (or Sharpe/MaxDD aren't better)


if __name__ == "__main__":
    main()
```